



**FACULTY OF TECHNOLOGY AND ENGINEERING  
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY  
AND RESEARCH**

**DEPARTMENT OF INFORMATION TECHNOLOGY**

**A.Y. 2026-27 Semester-3**

**LAB MANUAL**

**CSUC201 Fundamentals of Data Structures and Algorithms**



## Practical-4

### Practical AIM:

To implement core singly linked list operations and develop the ability to manipulate dynamic sequences through real-world problem scenarios.

### 4.1

#### Problem Definition:

A hospital manages a queue of patient tokens. New critical patients must be added to the front, routine patients are added to the end, and occasionally a patient with a priority number must be inserted at a specific position in the queue. Given a sequence of such operations, implement all three insertion types and print the final queue after each operation. Describe how your insertion at a specific position works. What does your solution do if the position given is greater than the current length of the queue? Is that handled explicitly or does it cause an error?

#### Output:

```
20
20 40
20 40 50
10 20 40 50
10 20 30 40 50
```

## Difficulty Faced:

The main difficulty was understanding how to perform three different types of insertion in a queue: inserting a critical patient at the front, a routine patient at the end, and inserting a priority patient at a specific position. It was also important to correctly shift elements when inserting at a particular position and handle invalid positions safely.

## Solution:

- Insert at Front: Add the new patient before the current first patient and update the front.
- Insert at End: Add the new patient after the current last patient.
- Insert at Specific Position: First check whether the position is valid. Then shift the elements after that position one place forward and insert the new patient there.
- Position greater than current length: The solution should handle this explicitly rather than causing an error. If the position is greater than the current queue length, display an appropriate message such as "Invalid position" and do not perform the insertion.

## Outcome:

Successfully implemented all three insertion operations and displayed the queue after each operation. The solution correctly handles insertion at the front, end, and a specified position. Invalid positions are checked explicitly, preventing an out-of-bounds error.

## 4.2

### Problem Definition:

The hospital also needs to remove a patient token from the queue when a patient leaves, print all remaining tokens from last to first for an end-of-day audit, and display the full queue from front to back at any point. Implement deletion by value, reverse printing, and forward traversal on the same queue from Problem 4.1a. Describe how your reverse printing works. Does it use any extra storage, or does it work purely by following the links? Can you think of another way to reverse-print without modifying the original queue?

### Output:

```
10
10 20
10 20 30
10 15 20 30
5 10 15 20 30
Invalid position!
5 10 15 20 30
```

### Difficulty Faced:

The main difficulty was implementing deletion by value and understanding how to traverse the queue in both directions. It was also necessary to make sure that deleting a patient does not break the links between the remaining nodes.

### Solution:

- Deletion by Value: Search the queue for the patient token. When found, remove that node and connect the previous node directly to the next node.

- Forward Traversal: Start from the front and follow the links until the end of the queue.
- Reverse Printing: Start from the last node and follow the links backward to print tokens from last to first.
- The reverse printing works by following the links, so the original queue is not modified.

## Outcome:

Successfully implemented deletion by value, forward traversal, and reverse printing on the same patient queue. The queue remains unchanged after reverse printing, and all remaining patient tokens can be displayed from both front-to-back and back-to-front.

## Key Questions:

1. [Problem 1] Inserting at a position beyond the current length of the list is an invalid operation. Should your solution silently insert at the end, report an error, or do something else entirely? Justify your choice and explain what assumption about the caller your choice makes.

Sol: The solution should report an error and not insert the node if the position is greater than the current length. This assumes that the caller provides a valid position and that invalid positions should be rejected rather than automatically changed.

1. [Problem 2] If the node to be deleted is the only node in the list, how many pointers need to change and what must they be set to? How is this case structurally different from deleting a node in the middle?

Sol: If the node is the only node, only the head pointer needs to change. It must be set to NULL. In a middle-node deletion, the previous node's next pointer must be changed to point to the next node.

**Github Link:**

<https://github.com/brindajivani/DSA/tree/main/Practical4>